

Ejercicio 1: Identificación de Roles

Objetivo: Reconocer los roles de Scrum y sus responsabilidades

Consigna: Para cada una de las siguientes situaciones, identifica qué rol de Scrum debería actuar y justifica tu respuesta:

- a) Un stakeholder quiere agregar una nueva funcionalidad urgente durante el Sprint
- b) El equipo no entiende claramente un requisito del Product Backlog
- c) Hay conflictos entre desarrolladores sobre la implementación técnica
- d) Se necesita decidir el orden de prioridad de las User Stories
- e) El equipo está teniendo problemas para cumplir con la Definition of Done

Roles disponibles: Product Owner, Scrum Master, Development Team

Solución:

- a) Rol que actúa: Product Owner (PO).
 - i) El PO es el único dueño del Product Backlog y el encargado de gestionar las expectativas de los stakeholders. Si alguien externo quiere meter un cambio, debe hablar con el PO. El equipo de desarrollo no debe aceptar requerimientos directamente de los stakeholders para no corromper el Sprint en curso.
- b) Rol que actúa: Product Owner (PO), pero asistido por el Equipo de Desarrollo.
 - i) El PO es el responsable de describir y aclarar las Historias de Usuario. Si el equipo tiene dudas sobre el "qué" o el "para qué" de una funcionalidad, debe acudir al PO para que la explique o detalle los criterios de aceptación.
- c) Rol que actúa: Development Team.
 - i) El equipo de desarrollo es autoorganizado y autónomo en el "cómo" se construye el software. Las decisiones técnicas y arquitectónicas dependen exclusivamente de ellos. Si el conflicto escala y afecta el ritmo de trabajo, el **Scrum Master** podría intervenir como facilitador para ayudarlos a llegar a un acuerdo, pero no para decidir la solución técnica.
- d) Rol que actúa: Product Owner (PO).

- i) El PO es el responsable maximizar el valor del producto. Por ende, es quien tiene la última palabra y la autoridad exclusiva para ordenar y priorizar los elementos del Product Backlog según el valor de negocio y las necesidades del cliente.
- e) Rol que actúa: Scrum Master (SM), junto al Equipo de Desarrollo.
 - i) El Scrum Master es el responsable de asegurar que el framework se entienda y se adopte correctamente, removiendo impedimentos y ayudando al equipo a mantener altos estándares de calidad. Si el equipo no llega a cumplir con el "Done" (terminado), el SM debe coachearlos para encontrar la causa raíz del problema.

Ejercicio 2: Artefactos de Scrum

Objetivo: Comprender el propósito y contenido de cada artefacto

Consigna: Completa la siguiente tabla identificando qué información NO debería estar en cada artefacto:

Solución:

Artefacto	¿Qué sí contiene?	¿Qué NO debería contener?
Product Backlog	Contiene la lista ordenada de todo lo que se conoce que es necesario en el producto, e incluye estimaciones de esfuerzo global en Story Points y prioridades claras definidas por el Product Owner.	No debería contener tareas técnicas ultra detalladas, asignación de nombres de desarrolladores específicos a cada ítem, ni fechas de entrega fijas y definitivas para cada requerimiento.
Sprint Backlog	Contiene el conjunto de elementos del Product Backlog seleccionados para el Sprint actual, junto con el plan detallado (tareas técnicas) para entregar el Incremento. También incluye el objetivo del Sprint (Sprint Goal) y el tiempo estimado restante para completar el trabajo.	No debería contener requerimientos o ideas futuras que no correspondan al Sprint en curso, historias de usuario que no hayan sido refinadas previamente, ni tareas de desarrollo ajenas al objetivo comprometido.
Increment	Contiene la suma de todos los elementos del Product Backlog completados durante el Sprint que cumplen rigurosamente con la Definition of Done (DoD). Es una porción de	No debería contener código a medio hacer o "casi terminado", funciones sin testear, ni mockups o maquetas visuales fijas en lugar de la funcionalidad de software real.

	software completamente funcional, testeada e integrada que representa un paso concreto hacia el Product Goal.	
--	---	--

Ejercicio 3: Eventos de Scrum - Verdadero o Falso

Objetivo: Identificar características correctas de los eventos Scrum

Marca V (Verdadero) o F (Falso) y justifica las falsas:

- La Sprint Planning puede durar hasta 8 horas para un Sprint de 4 semanas
 - **Verdadero:** Según la Guía Oficial de Scrum, la Sprint Planning está acotada a un máximo de 8 horas para un Sprint de un mes completo. Para Sprints de menor duración, este evento suele tomar proporcionalmente menos tiempo.
- En la Daily Scrum cada miembro debe reportar al Scrum Master
 - **Falso:** La Daily Scrum es una reunión de 15 minutos diseñada por y para el Equipo de Desarrollo con el fin de inspeccionar el progreso hacia el Sprint Goal y adaptar el plan de trabajo diario. El equipo se autoorganiza; no es una reunión para dar un reporte de estado al Scrum Master ni al Product Owner.
- La Sprint Review es solo para demostrar el producto al Product Owner
 - **Falso:** La Sprint Review es un evento colaborativo en el que participa todo el Equipo Scrum junto con los stakeholders (clientes o usuarios clave) invitados. Su propósito no es solo hacer una demo para el PO, sino inspeccionar el incremento terminado, recibir feedback del negocio y adaptar el Product Backlog.
- En la Sprint Retrospective se puede modificar el Sprint Backlog
 - **Falso:** La Sprint Retrospective ocurre al final del Sprint, momento en el cual el Sprint Backlog de ese ciclo ya cerró. El objetivo de este evento es evaluar cómo funcionó el equipo a nivel de procesos, herramientas y relaciones para planificar mejoras en la forma de trabajar del próximo Sprint.
- El Sprint puede cancelarse solo por decisión del Development Team
 - **Falso:** Únicamente el Product Owner cuenta con la autoridad formal para cancelar un Sprint antes de que se cumpla el tiempo límite. Esto ocurre solamente si el Objetivo del Sprint (Sprint Goal) queda obsoleto por un cambio drástico en el negocio o el mercado.

Ejercicio 4: Creación de Product Backlog

Objetivo: Practicar la redacción y priorización de User Stories

Contexto: Una empresa quiere desarrollar una aplicación móvil para delivery de comida.

Consigna:

1. Redacta 8 User Stories siguiendo el formato: "Como [usuario], quiero [funcionalidad] para [beneficio]"
2. Asigna Story Points usando la secuencia de Fibonacci (1,2,3,5,8,13,21)
3. Prioriza las historias del 1 al 8 (1 = máxima prioridad)
4. Define criterios de aceptación para las 3 historias de mayor prioridad

Solución:

1. Historia de Usuario 1 (Prioridad 1) - Registro e Inicio de sesión

Como cliente de la aplicación, **quiero** registrarme e iniciar sesión con mi correo electrónico y una contraseña, **para** poder mantener guardados mis datos de perfil e historial de pedidos.

Story Points: 3 puntos.

2. Historia de Usuario 2 (Prioridad 2) - Visualización del menú de restaurantes

Como cliente de la aplicación, **quiero** ver la lista de restaurantes disponibles junto con sus menús y precios actualizados, **para** poder elegir qué comida deseo ordenar.

Story Points: 5 puntos.

3. Historia de Usuario 3 (Prioridad 3) - Carrito de compras y pago

Como cliente de la aplicación, **quiero** agregar platos a un carrito de compras y pagar de forma segura con tarjeta de crédito, **para** concretar mi pedido de comida.

Story Points: 8 puntos.

4. Historia de Usuario 4 (Prioridad 4) - Seguimiento del pedido en tiempo real

Como cliente de la aplicación, **quiero** ver el estado de mi pedido en tiempo real (en preparación, en camino, entregado), **para** saber exactamente cuánto tardará en llegar.

Story Points: 8 puntos.

5. Historia de Usuario 5 (Prioridad 5) - Recepción de pedidos para el repartidor

Como repartidor de la empresa, **quiero** visualizar las solicitudes de entrega disponibles en mi zona con la dirección de destino, **para** aceptar pedidos y realizar las entregas eficientemente.

Story Points: 5 puntos.

6. Historia de Usuario 6 (Prioridad 6) - Gestión del menú para restaurantes

Como dueño de un restaurante asociado, **quiero** una interfaz para activar, desactivar o cambiar el precio de mis platos de comida, **para** mantener la oferta del menú actualizada.

Story Points: 5 puntos.

7. Historia de Usuario 7 (Prioridad 7) - Historial de pedidos anteriores

Como cliente de la aplicación, **quiero** acceder a un historial de mis compras pasadas, **para** poder repetir una orden que me gustó de forma rápida.

Story Points: 2 puntos.

8. Historia de Usuario 8 (Prioridad 8) - Calificación y reseñas

Como cliente de la aplicación, **quiero** calificar con estrellas y dejar un comentario sobre el restaurante y el repartidor, **para** compartir mi experiencia con la comunidad.

Story Points: 3 puntos.

Criterios de Aceptación para las 3 Historias de Mayor Prioridad

Criterios para la Historia de Usuario 1 (Registro e Inicio de sesión):

- **Escenario 1 (Registro exitoso):** Dado que el usuario ingresa un correo electrónico válido que no está registrado y una contraseña que cumple con los parámetros de seguridad, cuando presiona "Registrarse", el sistema debe crear la cuenta y redirigirlo a la pantalla principal.
- **Escenario 2 (Correo duplicado):** Dado que el usuario ingresa un correo electrónico que ya existe en el sistema, cuando presiona "Registrarse", el sistema debe bloquear la acción y mostrar el mensaje de error: "Este correo electrónico ya se encuentra registrado".

Criterios para la Historia de Usuario 2 (Visualización del menú de restaurantes):

- **Escenario 1 (Carga del menú):** Dado que el usuario selecciona un restaurante de la lista, cuando se abre su perfil, el sistema debe desplegar las categorías de comida de forma clara, mostrando foto, nombre del plato, descripción y precio.
- **Escenario 2 (Restaurante cerrado):** Dado que un restaurante está fuera de su horario de atención, cuando el usuario intente ver su menú, el sistema debe mostrar un cartel indicador de "Cerrado" y deshabilitar el botón de añadir productos.

Criterios para la Historia de Usuario 3 (Carrito de compras y pago):

- **Escenario 1 (Pago aprobado):** Dado que el usuario tiene productos en el carrito e ingresa los datos válidos de su tarjeta con fondos suficientes, cuando presiona "Confirmar Pago", el sistema debe procesar la transacción, mostrar un mensaje de éxito y vaciar el carrito de compras.
- **Escenario 2 (Tarjeta rechazada):** Dado que la pasarela de pagos rechaza la tarjeta por falta de fondos o datos incorrectos, cuando el usuario presiona "Confirmar Pago", el sistema debe notificar el error en pantalla de forma clara, manteniendo los productos guardados en el carrito para un nuevo intento.

Ejercicio 5: Planning Poker Simulation

Objetivo: Practicar la estimación colaborativa

Instrucciones:

- Forma grupos de 4-5 personas
- Cada grupo recibe las siguientes User Stories para estimar:
 - "Como usuario, quiero registrarme con email y contraseña"
 - "Como usuario, quiero recuperar mi contraseña olvidada"
 - "Como administrador, quiero generar reportes de ventas mensuales"
 - "Como usuario, quiero pagar con tarjeta de crédito de forma segura"

Proceso:

1. Cada miembro estima en privado (1,2,3,5,8,13,21 puntos)
2. Revelan estimaciones simultáneamente
3. Discuten diferencias significativas
4. Re-estiman hasta llegar a consenso

Solución:

1. Historia de Usuario: "Como usuario, quiero registrarme con email y contraseña"

- **Primera votación:** Dev 1: 3, Dev 2: 3, Dev 3: 2, Dev 4: 3, Dev 5: 5.
- **Discusión:** Dev 3 (voto menor - 2) argumenta que es una funcionalidad estándar del framework que ya viene casi cocinada. Dev 5 (voto mayor - 5) advierte que se deben incluir validaciones de seguridad de contraseña y encriptación en la base de datos, lo que suma complejidad.
- **Segunda votación (Consenso):** Todos los miembros votan **3 puntos**.
- **Estimación Final:** 3 Story Points.

2. Historia de Usuario: "Como usuario, quiero recuperar mi contraseña olvidada"

- **Primera votación:** Dev 1: 2, Dev 2: 3, Dev 3: 2, Dev 4: 2, Dev 5: 3.
- **Discusión:** Como las estimaciones están muy juntas entre 2 y 3 puntos, el equipo conversa brevemente. Coinciden en que, aunque es un flujo simple, requiere configurar el servicio de envío de correos automatizados (SMTP) e implementar un token con tiempo de expiración por seguridad.
- **Segunda votación (Consenso):** Todos los miembros votan **3 puntos**.
- **Estimación Final:** 3 Story Points.

3. Historia de Usuario: "Como administrador, quiero generar reportes de ventas mensuales"

- **Primera votación:** Dev 1: 5, Dev 2: 8, Dev 3: 5, Dev 4: 13, Dev 5: 5.
- **Discusión:** Dev 4 (voto mayor - 13) explica que calcular las ventas mensuales requiere cruzar datos de múltiples tablas (pedidos, descuentos, devoluciones, comisiones de repartidores) y teme que afecte la performance. Dev 1 y Dev 3 (votos menores - 5) sugieren que al ser un reporte asincrónico que se genera una vez al mes, la lógica matemática de backend es directa y no requiere interfaces en tiempo real. El equipo acuerda que es trabajoso pero no tan extremo.
- **Segunda votación (Consenso):** Todos los miembros votan **8 puntos**.
- **Estimación Final:** 8 Story Points.

4. Historia de Usuario: "Como usuario, quiero pagar con tarjeta de crédito de forma segura"

- **Primera votación:** Dev 1: 13, Dev 2: 8, Dev 3: 13, Dev 4: 21, Dev 5: 13.
- **Discusión:** Dev 4 (voto mayor - 21) plantea que la integración con la pasarela de pagos externa (como Stripe o Mercado Pago) requiere certificaciones de seguridad estrictas, manejo exhaustivo de errores de transacciones, contracargos y simulaciones en entornos de pruebas. El resto del equipo coincide en que el riesgo financiero y la complejidad de la API externa justifican elevar la estimación original.
- **Segunda votación (Consenso):** Todos los miembros votan **13 puntos**.
- **Estimación Final:** 13 Story Points.

Ejercicio 6: Sprint Planning Práctico

Objetivo: Planificar un Sprint completo

Contexto: Sprint de 2 semanas, equipo de 5 desarrolladores con velocidad promedio de 40 Story Points.

Product Backlog disponible:

- Historia A: 8 puntos - Prioridad Alta
- Historia B: 13 puntos - Prioridad Alta
- Historia C: 5 puntos - Prioridad Media

- Historia D: 8 puntos - Prioridad Media
- Historia E: 3 puntos - Prioridad Baja
- Historia F: 21 puntos - Prioridad Media

Tareas:

1. Selecciona las historias para el Sprint Backlog
2. Descompone la historia de mayor prioridad en tareas específicas
3. Define el Sprint Goal
4. Identifica posibles impedimentos y riesgos

Solución:

Primero damos una selección de historias para el Sprint Backlog para armar el Sprint Backlog, debemos respetar estrictamente el orden de prioridad y no superar la velocidad promedio del equipo, que es de 40 Story Points.

- Primero seleccionamos las de Prioridad Alta:
 - Historia B (13 puntos - Prioridad Alta) -> Acumulado: 13 puntos.
 - Historia A (8 puntos - Prioridad Alta) -> Acumulado: 21 puntos.
- Luego continuamos con las de Prioridad Media:
 - Historia D (8 puntos - Prioridad Media) -> Acumulado: 29 puntos.
 - Historia C (5 puntos - Prioridad Media) -> Acumulado: 34 puntos.

La Historia F tiene prioridad media pero vale 21 puntos. Si la sumamos ($34 + 21 = 55$), excederíamos ampliamente la capacidad de 40 puntos del equipo. La Historia E vale 3 puntos pero es de prioridad baja, por lo que queda fuera. Al equipo le quedan 6 puntos libres ($40 - 34$), por lo que decidimos cerrar el Sprint Backlog con las 4 historias seleccionadas para asegurar la calidad de la entrega.

- **Sprint Backlog Final:** Historias B, A, D y C.
- **Total de Story Points comprometidos:** 34 puntos.

Una vez hecho, descomponemos la historia de mayor prioridad en tareas específicas: La historia seleccionada con mayor prioridad y peso es la **Historia B (13 puntos - Prioridad Alta)**. Asumiendo que es la funcionalidad central más compleja de nuestro sistema, la desglosamos en las siguientes tareas técnicas para el equipo:

- Tarea 1: Analizar detalladamente los requerimientos y las reglas de negocio específicas de la interfaz (2 horas).
- Tarea 2: Diseñar la estructura y el modelado de las tablas correspondientes en la base de datos (4 horas).
- Tarea 3: Desarrollar los componentes de la interfaz de usuario (Frontend) asegurando el diseño responsivo (8 horas).

- Tarea 4: Programar la lógica de negocio y las APIs en el servidor (Backend) para el procesamiento de los datos (12 horas).
- Tarea 5: Integrar los componentes de Frontend y Backend (6 horas).
- Tarea 6: Escribir y ejecutar las pruebas unitarias y de integración para asegurar que cumple los criterios de aceptación (6 horas).

Definimos el Sprint Goal (Objetivo del Sprint): El Sprint Goal es la meta única que engloba el trabajo del equipo durante estas dos semanas. Se define como:

- "Lograr la implementación e integración completa de las funcionalidades críticas y de prioridad alta del sistema, garantizando un flujo estable de procesamiento de datos y consultas de nivel medio, listo para la validación del usuario final."

Finalmente, identificamos los posibles impedimentos y riesgos para anticiparnos a problemas que puedan afectar el cumplimiento del Sprint Goal, el equipo identifica:

- **Riesgo de subestimación:** Al ser la Historia B un elemento muy grande (13 puntos), existe el riesgo de que surja complejidad técnica oculta durante el desarrollo en backend que consuma más tiempo del planificado.
- **Dependencias externas:** Cuellos de botella en caso de requerir aprobaciones o aclaraciones de diseño por parte de los stakeholders sobre la marcha si las definiciones iniciales cambian.
- **Capacidad del equipo ajustada:** Al comprometer 34 puntos frente a una velocidad de 40, cualquier imprevisto de salud o ausencia de un desarrollador senior podría poner en riesgo la entrega de las historias de prioridad media (D y C).

Ejercicio 7: Simulación de Daily Scrum

Objetivo: Practicar la dinámica de la reunión diaria

Configuración: Grupos de 5-6 personas, uno actúa como Scrum Master

Escenario: Día 5 de un Sprint de 10 días. Cada desarrollador debe presentar:

- Qué hice ayer
- Qué haré hoy
- Qué impedimentos tengo

Tarjetas de situación (repartir al azar):

- "Terminé la tarea de login, hoy trabajaré en el registro de usuarios"
- "Ayer tuve problemas con la base de datos, necesito ayuda del DBA"
- "Completé las pruebas unitarias, hoy empiezo con la integración"
- "Estoy bloqueado esperando la API del sistema de pagos"
- "Terminé antes de lo esperado, puedo ayudar a alguien más"

Solución:

1. Intervención del Desarrollador 1 (Tarjeta: Login y Registro)

- **Qué hice ayer:** Terminé por completo la tarea de desarrollo del módulo de login de usuarios.
- **Qué haré hoy:** Hoy voy a comenzar a trabajar en la funcionalidad de registro de usuarios nuevos.
- **Qué impedimentos tengo:** No tengo ningún tipo de impedimento técnico ni bloqueos al día de hoy.

2. Intervención del Desarrollador 2 (Tarjeta: Base de datos)

- **Qué hice ayer:** Ayer estuve trabajando en el esquema de datos, pero lamentablemente tuve serios problemas de configuración con la base de datos.
- **Qué haré hoy:** Hoy mi plan es solucionar estos errores de conexión para poder avanzar con las tablas correspondientes.
- **Qué impedimentos tengo:** Estoy bloqueado en este punto; necesito la ayuda o asistencia del Administrador de Base de Datos (DBA) para destrabar los permisos del servidor.

3. Intervención del Desarrollador 3 (Tarjeta: Pruebas unitarias e Integración)

- **Qué hice ayer:** Completé de forma exitosa todas las pruebas unitarias para las funciones de backend que teníamos pendientes.
- **Qué haré hoy:** Hoy empiezo a trabajar directamente con las tareas de integración de los componentes en el entorno común.
- **Qué impedimentos tengo:** Ninguno por el momento, todo avanza de acuerdo a lo planificado.

4. Intervención del Desarrollador 4 (Tarjeta: API de pagos bloqueada)

- **Qué hice ayer:** Intenté avanzar con el diseño de la pasarela de pagos simulada.
- **Qué haré hoy:** Hoy planeaba programar la verificación de las transacciones con tarjeta.
- **Qué impedimentos tengo:** Estoy completamente bloqueado. Sigo esperando que el proveedor externo nos entregue la documentación técnica y las credenciales de la API del sistema de pagos. No puedo avanzar sin eso.

5. Intervención del Desarrollador 5 (Tarjeta: Tareas adelantadas y soporte)

- **Qué hice ayer:** Estuve trabajando en mis requerimientos asignados y logré terminarlos todos antes de lo esperado.
- **Qué haré hoy:** Dado que no tengo tareas críticas pendientes en mi tablero, me pondré a disposición del equipo.

- **Qué impedimentos tengo:** Ninguno. De hecho, como estoy libre, me ofrezco para ayudar al Desarrollador 2 a revisar el problema de la base de datos o acompañar el reclamo de la API de pagos.

Cierre del Scrum Master: El Scrum Master toma nota de los dos impedimentos importantes mencionados (la necesidad del DBA para el Desarrollador 2 y el reclamo de la API de pagos externa para el Desarrollador 4). Asigna al Desarrollador 5 para brindar soporte inmediato en el problema de base de datos y se compromete a gestionar personalmente la llamada con el proveedor del sistema de pagos para liberar el bloqueo técnico antes de que termine el día.

Ejercicio 8: Sprint Review y Retrospective

Objetivo: Practicar la inspección y adaptación

Parte A - Sprint Review: El equipo logró completar 35 de 40 Story Points comprometidos.

Prepara:

1. Agenda de la reunión (duración: 2 horas)
2. Métricas a presentar
3. Preguntas para los stakeholders
4. Plan para las historias no completadas

Parte B - Sprint Retrospective: Usando la técnica "Start, Stop, Continue", identifica:

- Qué debe empezar a hacer el equipo
- Qué debe dejar de hacer
- Qué debe continuar haciendo

Situaciones a analizar:

- Algunas tareas tomaron más tiempo del estimado
- Un miembro del equipo estuvo enfermo 3 días
- Hubo cambios de último momento en los requisitos
- La colaboración entre front-end y back-end mejoró

Solución:

Parte A - Sprint Review

1. Agenda de la reunión (Duración total: 2 horas)

- **Apertura y Bienvenida (10 minutos):** El Scrum Master da la bienvenida a los stakeholders y al equipo, y repasa brevemente el Objetivo del Sprint (Sprint Goal).

- **Presentación del Estado del Incremento (15 minutos):** El Product Owner explica qué elementos del Product Backlog se completaron ("Done") y cuáles no se alcanzaron a terminar.
- **Demostración del Producto (50 minutos):** El Equipo de Desarrollo realiza una demo en vivo mostrando las funcionalidades reales de las historias de usuario terminadas (35 Story Points logrados). Los stakeholders prueban el incremento.
- **Feedback de los Stakeholders (30 minutos):** Espacio abierto para que los clientes expresen sus opiniones, necesidades actuales del mercado y posibles cambios en el orden del día.
- **Revisión del Backlog y Cierre (15 minutos):** El Product Owner y los stakeholders analizan las proyecciones de fechas de entrega y revisan las prioridades del Product Backlog para el siguiente Sprint.

2. Métricas a presentar

- **Velocidad del Sprint (Sprint Velocity):** 35 Story Points completados frente a los 40 comprometidos (87.5% de efectividad).
- **Gráfico de Quemado (Burndown Chart):** Curva que muestra cómo fue el ritmo diario de quema de tareas a lo largo de las dos semanas.
- **Defectos reportados (Bugs):** Cantidad de incidentes encontrados y resueltos durante la fase de integración dentro del Sprint.

3. Preguntas para los stakeholders

- ¿Las pantallas y flujos que les mostramos en la demo cubren la experiencia de usuario que se imaginaban para el negocio?
- Teniendo en cuenta lo que vieron hoy, ¿consideran que hay alguna funcionalidad en el Product Backlog que haya perdido prioridad o que debamos agregar con urgencia?

4. Plan para las historias no completadas

- Las historias que suman los 5 Story Points restantes no terminados vuelven de manera automática al Product Backlog.
- El Product Owner reevaluará su prioridad. Si siguen siendo importantes para el negocio, se refinarán para entender por qué se trabaron y el equipo las volverá a considerar en la Sprint Planning del próximo ciclo.

Parte B - Sprint Retrospective

Utilizando la técnica **"Start, Stop, Continue"**, el equipo analiza las situaciones planteadas durante el Sprint (tareas demoradas, baja por enfermedad, cambios de requisitos de último momento y mejoras en la colaboración):

- **START (Qué debe empezar a hacer el equipo):**
 - **Empezar a definir un plan de contingencia (cross-training):** Ante la situación del miembro que estuvo enfermo 3 días, el equipo debe empezar a compartir

más el conocimiento técnico para que otros puedan tomar tareas críticas y evitar cuellos de botella.

- **Empezar a proteger el Sprint Backlog:** Frente a los cambios de último momento en los requisitos por parte de los stakeholders, el Scrum Master debe empezar a capacitar a la organización para que entienda que el alcance del Sprint no se modifica una vez que inició.
- **STOP (Qué debe dejar de hacer el equipo):**
 - **Dejar de subestimar las tareas técnicas complejas:** Como algunas tareas tomaron más tiempo del estimado, el equipo debe dejar de poner puntajes "a ojo" rápidos y dedicarle más tiempo al análisis de riesgos durante la planificación.
- **CONTINUE (Qué debe continuar haciendo el equipo):**
 - **Continuar con los canales de comunicación cruzados:** Se debe continuar potenciando las dinámicas de emparejamiento (Pair Programming) y reuniones de sincronización corta, ya que la colaboración entre front-end y back-end mejoró notablemente en este ciclo.

Ejercicio 9: Manejo de Impedimentos

Objetivo: Identificar y resolver bloqueos típicos de Scrum

Situaciones problema:

1. El Product Owner no está disponible para aclarar dudas sobre requisitos
2. Un desarrollador senior está sobrecargado y se convierte en cuello de botella
3. Las herramientas de desarrollo están fallando constantemente
4. Hay conflictos interpersonales en el equipo
5. Un stakeholder presiona para agregar funcionalidades no planificadas

Para cada situación:

- Identifica el tipo de impedimento
- Propone 2-3 estrategias de resolución
- Define quién debería actuar
- Establece un plan de seguimiento

Solución:

Situación 1: El Product Owner no está disponible para aclarar dudas sobre requisitos.

Tipo de Impedimento: Impedimento de Proceso / Bloqueo por Comunicación Organizacional.

Quién debería actuar: El Scrum Master.

- **Estrategias de Resolución:**

1. El Scrum Master debe reunirse con el Product Owner para concientizarlo sobre el impacto de su ausencia (retrasos en el Sprint).
2. Establecer bloques de tiempo fijos diarios (por ejemplo, 30 minutos después de la Daily) exclusivos para atender dudas del equipo.
3. Acordar con el Product Owner delegar la toma de decisiones menores en el Analista o en el Líder Técnico durante sus horas de ausencia.

Plan de Seguimiento: Monitorear en las siguientes Daily Meetings si las tareas dependientes del PO se destraban rápido o si el equipo sigue esperando definiciones.

Situación 2: Un desarrollador senior está sobrecargado y se convierte en cuello de botella.

Tipo de Impedimento: Impedimento de Equipo / Dependencia Técnica de Personal (Key-man risk).

Quién debería actuar: El Scrum Master y el Equipo de Desarrollo de forma conjunta.

- **Estrategias de Resolución:**

1. Detener la asignación de nuevas tareas complejas al desarrollador senior durante el Sprint en curso.
2. Implementar dinámicas de Pair Programming (programación de a pares) para que el senior resuelva tareas complejas acompañado por un desarrollador junior/semi-senior, transfiriendo conocimiento.
3. Asegurar que el senior dedique tiempo a documentar procesos críticos antes de seguir tirando código.

Plan de Seguimiento: Revisar visualmente el tablero Kanban en las Daily Meetings para verificar si la acumulación de tareas pendientes en la columna del desarrollador senior disminuye de forma progresiva.

Situación 3: Las herramientas de desarrollo están fallando constantemente.

Tipo de Impedimento: Impedimento Técnico / Tecnológico.

Quién debería actuar: El Scrum Master para la gestión y el Equipo de Desarrollo (o área de infraestructura/IT correspondientes) para la ejecución.

- **Estrategias de Resolución:**

1. Registrar formalmente los incidentes y cuantificar las horas de trabajo perdidas por el equipo debido a los fallos técnicos.
2. Escalar el problema al área de soporte técnico o infraestructura de la empresa exigiendo una revisión prioritaria de los servidores o licencias.
3. Buscar alternativas de contingencia temporales (como entornos de desarrollo locales o herramientas de código abierto equivalentes) para no frenar la marcha.

Plan de Seguimiento: Validar diariamente si se repiten los cortes técnicos y comprobar en la Retrospectiva si se estabilizó el ambiente de trabajo.

Situación 4: Hay conflictos interpersonales en el equipo.

Tipo de Impedimento: Impedimento Humano / Relacional.

Quién debería actuar: El Scrum Master (actuando como mediador y coach).

- **Estrategias de Resolución:**

1. Mantener reuniones uno a uno (1-on-1) de manera privada con las partes involucradas para escuchar sus posturas sin juzgar.
2. Organizar una sesión de mediación neutral enfocada en resolver el problema técnico o de proceso subyacente, dejando de lado los ataques personales.
3. Recordar y reforzar los Valores de Scrum establecidos (Respeto, Apertura, Compromiso) mediante dinámicas de integración de equipo (Team Building).

Plan de Seguimiento: Observar el comportamiento, tono y la colaboración del equipo en las ceremonias cotidianas durante las próximas dos semanas para garantizar un clima laboral sano.

Situación 5: Un stakeholder presiona para agregar funcionalidades no planificadas.

Tipo de Impedimento: Impedimento Externo / Ruido del Alcance (Scope Creep).

Quién debería actuar: El Product Owner (como protector del producto) respaldado por el Scrum Master (como protector del proceso).

- **Estrategias de Resolución:**

1. El Product Owner debe interceptar al stakeholder y explicarle firmemente que el Sprint Backlog actual está cerrado y comprometido para resguardar la calidad.
2. El Product Owner debe tomar el requerimiento del stakeholder, redactarlo como una nueva Historia de Usuario y colocarlo en el Product Backlog general para ser priorizado en los futuros Sprints.
3. El Scrum Master capacita al stakeholder acerca del framework ágil para que comprenda cómo y cuándo proponer cambios formalmente.

Plan de Seguimiento: Asegurarse de que ninguna de esas solicitudes "informales" ingrese al tablero del Sprint actual y corroborar en la Sprint Review si el requerimiento fue considerado en el Product Backlog.

Ejercicio 10: Definición de Terminado (Definition of Done)

Objetivo: Crear criterios de calidad compartidos

Contexto: Equipo desarrollando una aplicación web de e-commerce

Consigna: Crea una Definition of Done que incluya:

- Criterios técnicos
- Criterios de calidad
- Criterios de documentación
- Criterios de validación

Considera estos aspectos:

- Código revisado por pares
- Pruebas automatizadas
- Documentación técnica
- Validación del Product Owner
- Despliegue en ambiente de testing
- Criterios de performance
- Accesibilidad
- Seguridad

Solución:

Para que una Historia de Usuario (User Story) se considere oficialmente "Terminada" (Done) en nuestra aplicación web de e-commerce, debe cumplir rigurosamente con los siguientes criterios establecidos por el equipo:

1. Criterios Técnicos

- El código fuente debe estar completamente escrito, limpio de comentarios temporales o código muerto, y refactorizado bajo los estándares del equipo.
- Se debe realizar una revisión de código por pares (Peer Review / Pull Request) con la aprobación obligatoria de al menos un desarrollador senior antes de fusionar a la rama principal (main/master).
- El código debe compilar correctamente en el servidor de integración continua sin generar advertencias ni errores críticos.

2. Criterios de Calidad

- Se deben diseñar y ejecutar pruebas automatizadas (unitarias y de integración), alcanzando un mínimo del 80% de cobertura (code coverage).
- La funcionalidad debe pasar las pruebas de seguridad básicas, garantizando la encriptación de datos sensibles del cliente (como contraseñas y tarjetas de crédito).

- Se deben cumplir los criterios de performance: el tiempo de carga de la página del e-commerce no debe superar los 2 segundos en conexiones estándar.
- Cumplir con las pautas de accesibilidad web (WCAG) mínimas para asegurar que la plataforma pueda ser navegada por personas con discapacidades.

3. Criterios de Documentación

- La documentación técnica del sistema (diagramas de arquitectura, modelos de base de datos o endpoints de la API) debe estar actualizada en el repositorio central (Wiki o Readme).
- Cualquier nueva variable de entorno o configuración del servidor debe quedar registrada de forma clara para el equipo de infraestructura.

4. Criterios de Validación

- La historia de usuario debe ser desplegada de forma exitosa en el ambiente de testing o preparación (Staging).
- Se deben ejecutar de forma satisfactoria los casos de prueba manuales y de regresión para asegurar que no se rompió ninguna funcionalidad previa del e-commerce.
- Validación final del Product Owner: El PO debe verificar que la funcionalidad cumple con todos y cada uno de los criterios de aceptación específicos redactados originalmente en la historia de usuario.

Preguntas de Cierre:

1. ¿Cuáles fueron los principales desafíos al aplicar Scrum en estos ejercicios?
2. ¿Qué beneficios observaste en el trabajo colaborativo?
3. ¿Cómo adaptarías Scrum para un proyecto real en tu área de trabajo?
4. ¿Qué aspectos de Scrum consideras más difíciles de implementar?

1). Yo creo que uno de los principales desafíos fue lograr un consenso real en el Planning Poker: porque gestionar las discrepancias significativas de criterios entre desarrolladores durante las simulaciones de votación exigió un ejercicio profundo de comunicación, argumentación técnica y escucha activa para llegar a un acuerdo unificado.

2). Observé un enfoque compartido en la calidad: La creación y el respeto mutuo de una Definition of Done (DoD) colectiva asegura que todo el equipo comparta la misma vara de exigencia técnica, eliminando las suposiciones individuales de lo que significa que una tarea esté realmente "hecha".

3). Principalmente organizaría sprints cortos y enfocados: Establecería ciclos fijos de 2 semanas para mantener un ritmo de entregas constante, mitigar la incertidumbre del mercado y poder recibir feedback frecuente de los usuarios o clientes reales.

4). El más difícil de implementar sin dudas es gestionar la interferencia de los Stakeholders: En el día a día de las organizaciones, uno de los retos más complejos es lograr que los clientes o gerentes respeten el blindaje del Sprint Backlog y canalicen sus urgencias a través del Product Owner sin presionar directamente a los desarrolladores con cambios de último momento.